PyFK: A Fast MPI and CUDA Accelerated Python Package for Calculating Synthetic Seismograms Based on the Frequency–wavenumber Method

Ziyi Xi[1], Jiaqi Li[2], Min Chen[1,3], and Songqiao Shawn Wei[3]

[1]Michigan State University, Department of Computational Mathematics, Science and Engineering, East Lansing, MI, United States
[2]University of California Los Angeles, Earth, Planetary, and Space Sciences, Los Angeles, United States
[3]Michigan State University, Department of Earth and Environmental Sciences, East Lansing, MI, United States

CMSE

Department of Earth and Environmental Sciences
MICHIGAN STATE UNIVERSITY

ICER

XSEDE
Extreme Science and Engineering Discovery Environment

NSF

Introduction

Some software to calculate synthetic seismograms

Reflectivity/Wavenumber integration for 1D layered Earth	
FK (L Zhu)	Calculate synthetic seismograms in layered isotropic models, using frequency-wavenumber method
QSEIS (R Wang)	Calculating synthetic seismograms based on a layered viscoelastic half-space earth model
Normal modes summation for 1D layered spherical Earth	
QSSP (R Wang)	Calculating complete synthetic seismograms of a spherical earth using the normal mode theory
Finite Difference Methods	
SW4 (Petersson et al.)	3D Finite-Difference Seismic Wave Simulation (4th order)
Spectral Element Methods	
AxiSEM (Nissen-Meyer et al.)	A parallel spectral-element method to solve 3D wave propagation in a sphere with axisymmetric or spherically symmetric visco-elastic, acoustic, anisotropic structures
Specfem3D_GLOBE (Komatitsch and Tromp, with many other authors)	Simulates seismic wave propagation in a 3D heterogeneous medium, using spectral element method (spherical coordinate system)

Why PyFK? (for 1D layered model)

- Implemented in pure Python
- Fast synthetic seismograms calculator
- Accelerated by Cython, MPI, and CUDA
- A seamless integration with existing packages, especially Obspy

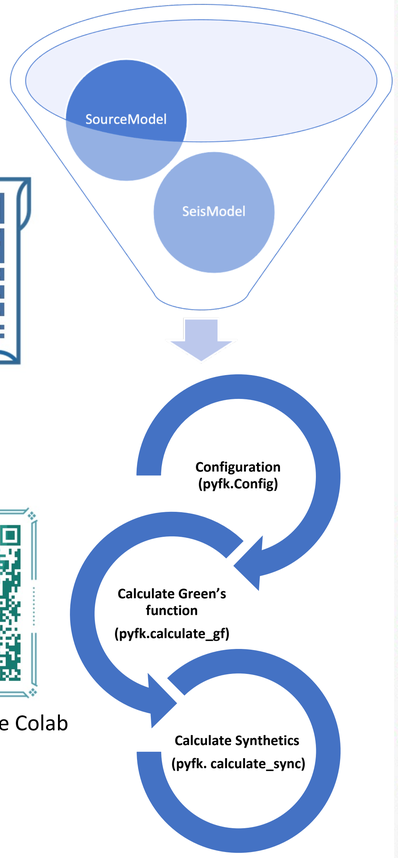
The frequency-wavenumber method

Historically, Haskell (1963, 1964) and Harkrider (1964) use the propagator matrix technique, which studies the matrix formalism of wave propagation in a multi-layered medium to derive the solution of the surface displacement due to a point source. On the other hand, Zhu and Rivera (2002) redefine the propagator matrix and find the static solution can be a special case of the dynamic solution. In the meantime, Prof. Zhu wrote the package FK to calculate the 1D layered model synthetics, which is then widely used by the seismology community.

By setting up a cylindrical coordinate system, the displacement in a vertically heterogeneous medium can be expanded in terms of three orthogonal vectors (Takeuchi and Saito 1972):

$$\mathbf{u}(r,\theta,z,t) = \frac{1}{2\pi}\sum_{m=0,\pm1,\ldots}\int e^{-i\omega t}d\omega\int_0^\infty kdk\left(U_z\mathbf{R}_m^k + U_r\mathbf{S}_m^k + U_\theta\mathbf{T}_m^k\right)$$

Here the integration of ω is an inverse Fourier transform. The integration of the wavenumber k must be done numerically. R, S and T are Bessel functions, while U_z, U_θ, and U_t can be solved by the propagator matrix technique.



PyFK on GitHub

Try PyFK on Google Colab

```python
# import essencial packages
from pyfk import (SourceModel, SeisModel, Configcalculate_gf,
        generate_source_time_function, calculate_sync)
from obspy import read_events
import numpy as np

########### part1 configurations ###################
# the source configuration, depth 16.5 km, double couple
source_prem = SourceModel(sdep=16.5, srcType="dc")
# assume the model file prem.txt is following FK format
prem_data = np.loadtxt("prem.txt")
# the model configuration can be defined accordingly
model_prem = SeisModel(model=prem_data)
# the simulation configuration, 512 points with delta=0.1,
# epicenter distance in 10, 20, 30 degree
config_prem = Config(
        model=model_prem,
        source=source_prem,
        npt=512,
        dt=0.1,
        degree=True,
        receiver_distance=np.arange(10, 40, 10))

########### part2 Green's function ###################
# gf is a list of Obspy's Trace, containing Green's function's
# data and header information
gf = calculate_gf(config_prem)

########### part3 Synthetics ###################
# assume test_gcmt_path is a Global CMT Solution file
# we can update the source model based on the source mechanism
event=obspy.read_events(test_gcmt_path)[0]
source_prem.update_source_mechanism(event)
# sync_result is an Obspy's Stream storing the waveform, 30 is azimuth
sync_result = calculate_sync(gf, config_prem, 30, source_time_function)
# we can further process the synthetics using Obspy
sync_result.detrend("linear")
sync_result.taper(max_percentage=0.05, type='hann')
sync_result.filter("bandpass", freqmin=1/8, freqmax=0.5,
                corners=2, zerophase=True)
```

Strong scaling test for the MPI mode

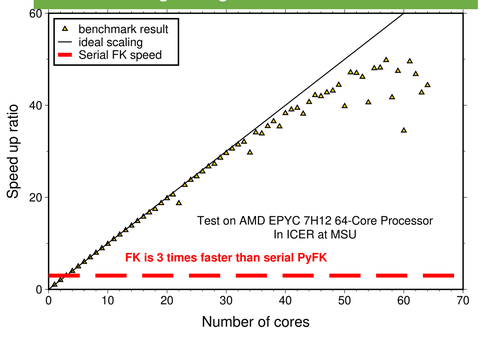


Fig. 1 The figure shows that with less than 50 processes, there is almost a linear speed-up ratio which is identical to the ideal case. For processes larger than 50, the speed-up ratio is almost consistent with lesser improvement. It suggests the optimized number of processes is 50.

Weak scaling test for the MPI mode

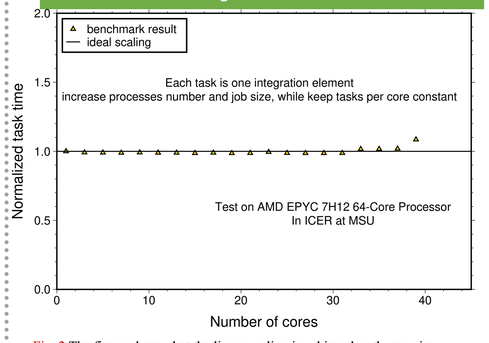


Fig. 2 The figure shows that the linear scaling is achieved as the run time stays constant while the workload is increased. It implies no bottleneck such as from IO that limits the parallel computing's efficiency.

Performance on GPU

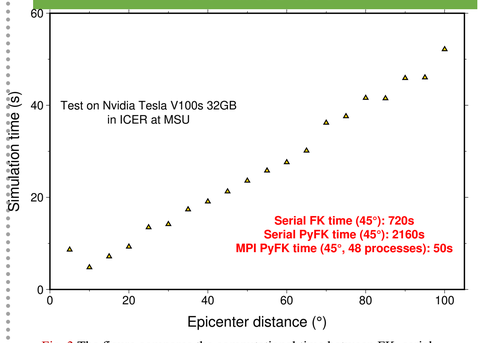
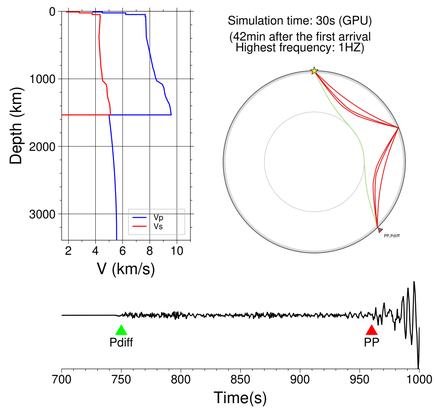


Fig. 3 The figure compares the computational time between FK, serial version of PyFK, PyFK with MPI, and PyFK with CUDA. The model is a layered PREM model (around 50 layers, Dziewonski and Anderson 1981). The depth of the source is 16.5 km, with 5120 points and 0.1s time intervals.

Example: Marsquake simulation



The waveform simulation for an Marsquake. Scientists already know more about the Aug. 25 quakes: The magnitude 4.2 event occurred about 5,280 miles (8,500 kilometers) from InSight – the most distant temblor the lander has detected so far. (cited from https://www.jpl.nasa.gov/news/nasas-insight-finds-three-big-marsquakes-thanks-to-solar-panel-dusting)

The Mars model is cited from Stähler et al. (2021).

Implementations

We propose the following implementations of PyFK:

- Calculate the Green's function and the synthetic seismograms for the 1D layered model
- Invert for the 1D layered model with the Bayesian Monte Carlo method. It will be much more efficient with speed up from MPI and CUDA.
- PyFK is also a good tool for educational purposes. It is easy to use, straightforward to visualize, and friendly to Python users.

Use cases:

- MCMTpy (Yin et al.): a Python package designed for seismic source study. It provides functionality for focal mechanism inversion and source parameters analysis.

Acknowledgement

This work is supported by NSF grant 1802247. We thank the High-Performance Computing Center (HPCC) at Michigan State University, the Extreme Science and Engineering Discovery Environment (XSEDE supported by NSF grant ACI-1053575).

S15E-0288, 13 December 2021, 17:00 pm – 19:00 pm

www.PosterPresentations.com